

Hadoop and MapReduce

Simplified Data Processing on Large Clusters

KTP 2023

Outline

- Problem Statement / Motivation
- An Example Problem
- MapReduce vs Hadoop
- GFS / HDFS
- MapReduce Fundamentals
- Example Code
- Workflows
- Conclusion / Questions
- Extra

Problem Statement / Motivation - Why MapReduce?

- Before MapReduce
 - Large Concurrent Systems
 - Grid Computing
 - Rolling Your Own Solution
- Considerations
 - Threading is hard!
 - How do you scale to more machines?
 - How do you handle machine failures?
 - How do you facilitate communication between nodes?

Problem Statement / Motivation – Problem Scope

- *MapReduce* is a programming model for data processing
- The power of MapReduce lies in its ability to scale to 100s or 1000s of computers, each with several processor cores
- How large an amount of work?
 - Web-Scale data on the order of 100s of GBs to TBs or PBs
 - It is likely that the input data set will not fit on a single computer's hard drive
 - Hence, a distributed file system (e.g., Google File System- GFS) is typically required

Problem Statement / Motivation – commodity cluster

- MapReduce is designed to efficiently process large volumes of data by connecting many commodity computers together to work in parallel
- A *theoretical* 1000-CPU machine would cost a very large amount of money, far more than 1000 single-CPU or 250 quad-core machines
- MapReduce ties smaller and more reasonably priced machines together into a single cost-effective *commodity cluster*

An Example Problem

- The typical example of, “Word Count”
- The input of this program is a volume of raw text, of unspecified size (could be KB, MB, TB, it doesn’t matter!)
- The output is a list of words, and their occurrence count.
- Assume that words are split correctly, ignoring capitalization and punctuation.

An Example Program – (Cont.)

- Example

“The doctor went to the store”
- Manually count:
 - The, 2
 - Doctor, 1
 - Went, 1
 - To, 1
 - Store, 1

MapReduce vs Hadoop

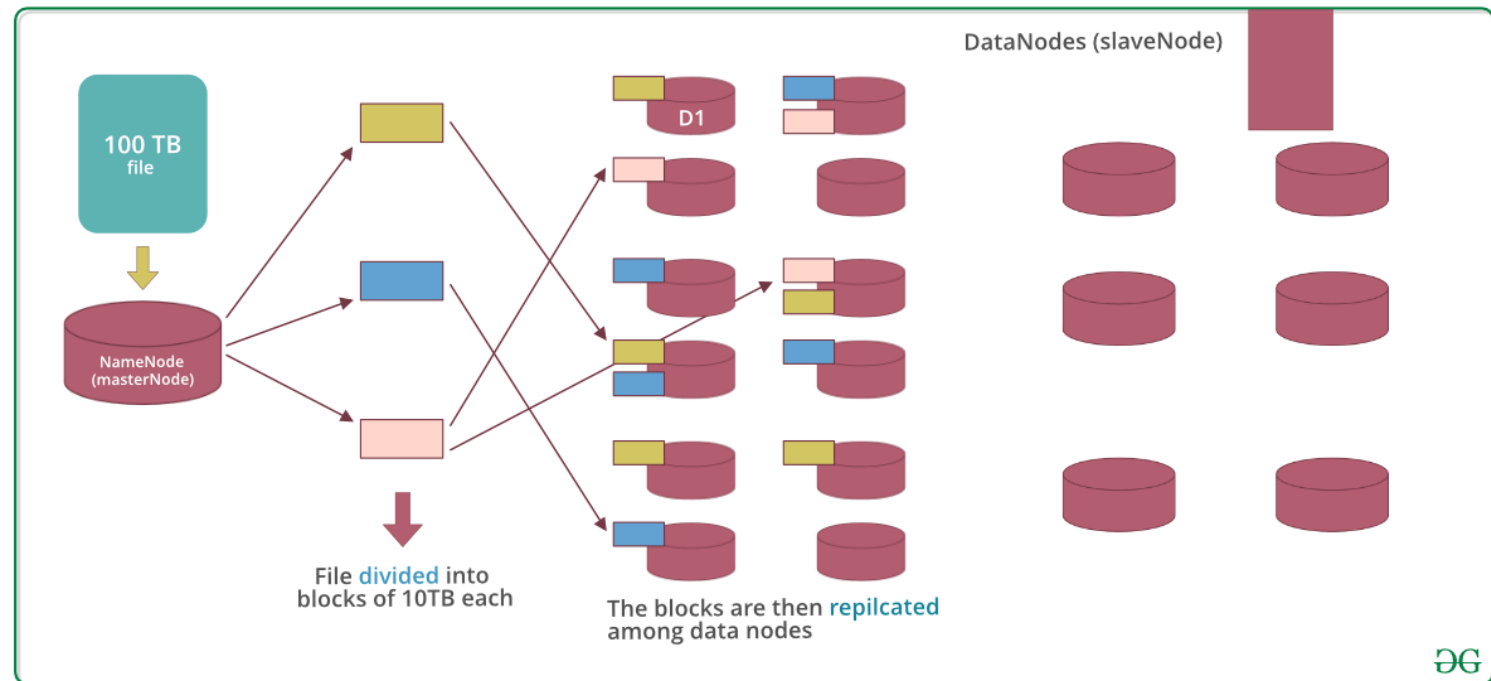
- Hadoop is an open source distributed processing framework that manages data processing and storage for big data applications in scalable clusters of computer servers
- HDFS is a storage unit of Hadoop
- MapReduce is a processing tools of Hadoop
- Google's implementation is written in C++ (GFS)
- Apache Hadoop is written in Java (HDFS)
- MapReduce can be query by Hive, it is similar to SQL but not the standard SQL

GFS/HDFS

- A few concepts are key to MapReduce:
 - Google File System (GFS) and Hadoop Distributed File System (HDFS) are essentially distributed filesystems
 - Are fault tolerant through replication
 - Allows data to be local to computation

Apache HDFS

- HDFS = Hadoop Distributed File System
- Designed to reliably store very large files across machines in a large cluster
- Stores each file as a sequence of blocks





Apache Hadoop Ecosystem



Ambari

Provisioning, Managing and Monitoring Hadoop Clusters



Sqoop

Data Exchange



Oozie

Workflow



Pig

Scripting



Mahout

Machine Learning

R Connectors

Statistics



Hive

SQL Query



Hbase

Columnar Store



Flume

Log Collector

Zookeeper

Coordination



YARN Map Reduce v2

Distributed Processing Framework

HDFS

Hadoop Distributed File System



Major Components of MapReduce

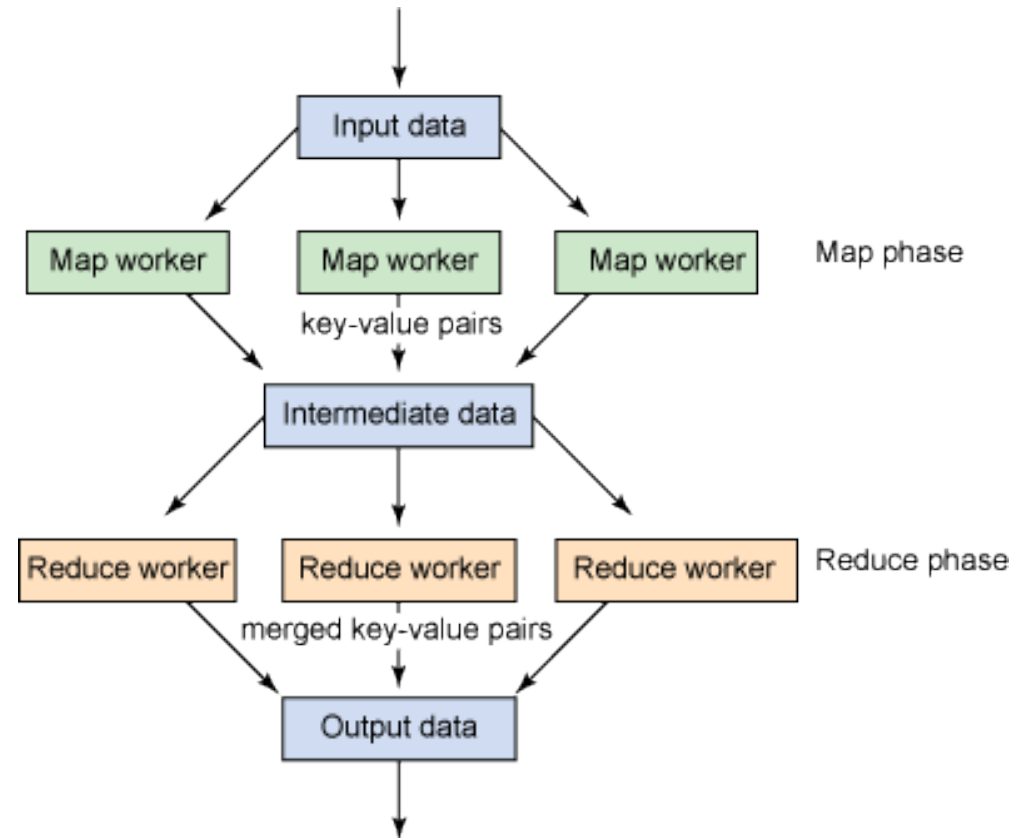
- User Components:

- Mapper
- Reducer
- Combiner (Optional)
- Partitioner (Optional) (Shuffle)
- Writable(s) (Optional)

- System Components:

- Master
- Input Splitter*
- Output Committer*

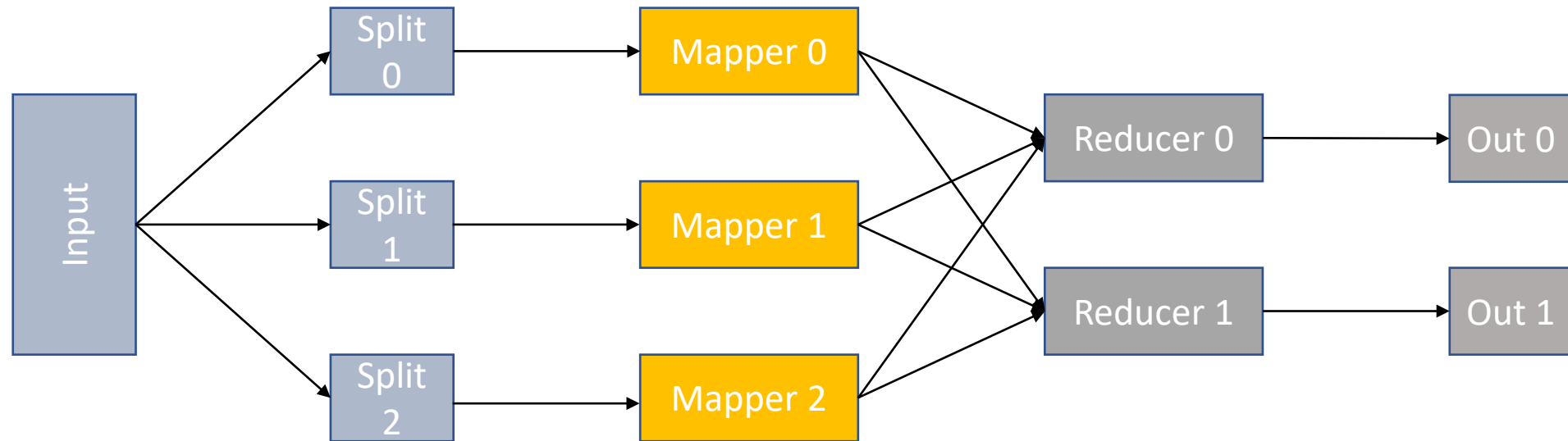
* You can use your own if you really want!



Key Notes

- Mappers and Reducers are typically single threaded and deterministic
 - Determinism allows for restarting of failed jobs, or speculative execution
- Need to handle more data? Just add more Mappers/Reducers!
 - No need to handle multithreaded code
 - Since they're all independent of each other, you can run (almost) arbitrary number of nodes
- Mappers/Reducers run on arbitrary machines.
 - A machine typically multiple map and reduce slots available to it, typically one per processor core
- Mappers/Reducers run entirely independent of each other
 - In Hadoop, they run in separate JVMs

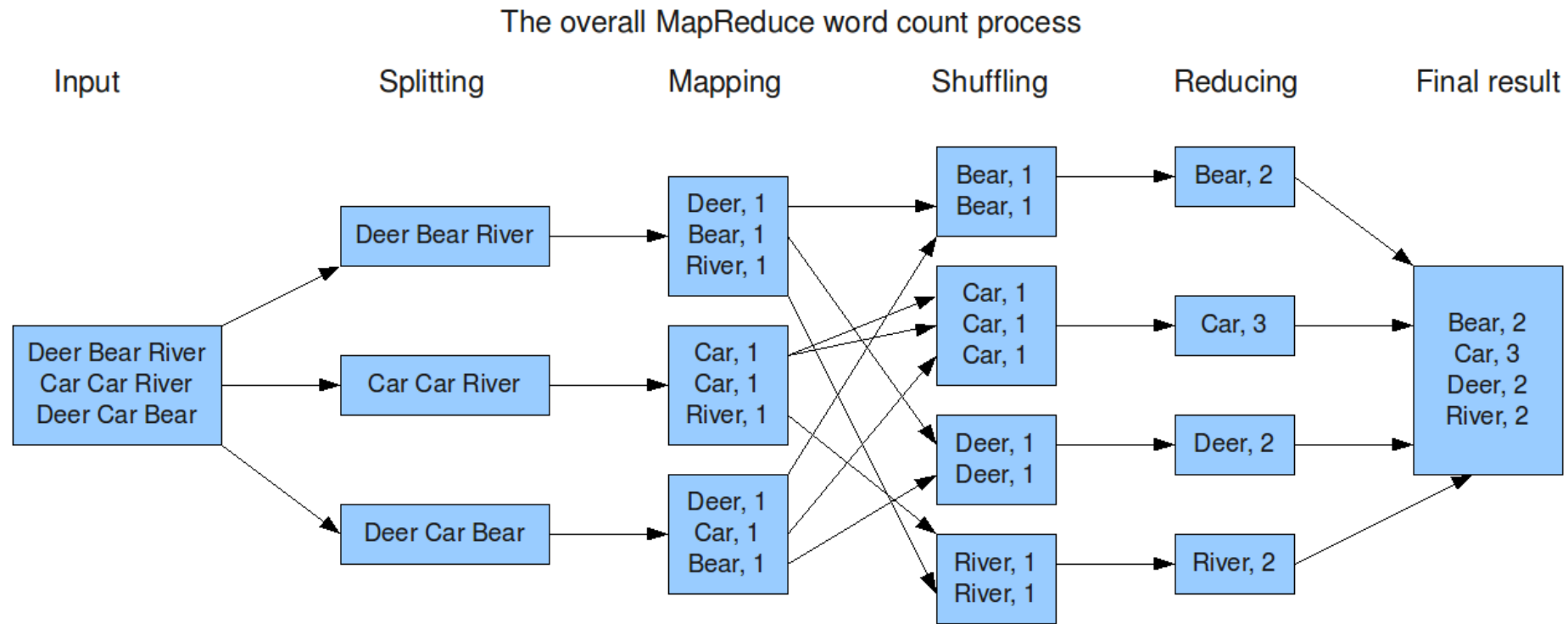
Data Flow



Basic Concepts

- All data is represented in key value pairs of an arbitrary type
- Data is read in from a file or list of files, from HDFS
- Data is chunked (batch) based on an input split, A typical chunk is 64MB (more or less can be configured depending on your use case)
- Mappers read in a **chunk** of data
- Mappers emit (write out) a set of data, typically derived from its input
- Intermediate data (the output of the mappers) is split to a number of reducers
- Reducers receive each key of data, along with **ALL** of the values associated with it (this means each key must always be sent to the same reducer)
 - Essentially, <key, set<value>>
- Reducers emit a set of data, typically reduced from its input which is written to disk

Remember, Word Count?



Splitting

- Input splitter is responsible for splitting your input into multiple chunks
- These chunks are then used as input for your mappers
- Splits on logical boundaries. The default is 64MB per chunk
 - Depending on what you're doing, 64MB might be a LOT of data! You can change it
- Typically, you can just use one of the built in splitters, unless you are reading in a specially formatted file

Mapping

- Mapper reads in input pair $\langle K, V \rangle$ (a section as split by the input splitter)
- Outputs a pair $\langle K', V' \rangle$
- Ex. For our Word Count example, with the following input: “The teacher went to the store. The store was closed; the store opens in the morning. The store opens at 9am.”
- The output would be:
 - $\langle \text{The}, 1 \rangle \langle \text{teacher}, 1 \rangle \langle \text{went}, 1 \rangle \langle \text{to}, 1 \rangle \langle \text{the}, 1 \rangle \langle \text{store}, 1 \rangle \langle \text{the}, 1 \rangle \langle \text{store}, 1 \rangle \langle \text{was}, 1 \rangle \langle \text{closed}, 1 \rangle \langle \text{the}, 1 \rangle \langle \text{store}, 1 \rangle \langle \text{opens}, 1 \rangle \langle \text{in}, 1 \rangle \langle \text{the}, 1 \rangle \langle \text{morning}, 1 \rangle \langle \text{the}, 1 \rangle \langle \text{store}, 1 \rangle \langle \text{opens}, 1 \rangle \langle \text{at}, 1 \rangle \langle \text{9am}, 1 \rangle$

Partitioner (Shuffler)

- Decides which pairs are sent to which reducer
- Default is simply:
 - `Key.hashCode() % numOfReducers`
- User can override to:
 - Provide (more) uniform distribution of load between reducers
 - Some values might need to be sent to the same reducer
 - Ex. To compute the relative frequency of a pair of words <W1, W2> you would need to make sure all of word W1 are sent to the same reducer
 - Binning of results

Reducing

- Reducer accepts the Mapper output, and collects values on the key
 - All inputs with the same key *must* go to the same reducer!
- Input is typically sorted, output is output exactly as is
- For our example, the reducer input would be:
 - <The, 1> <teacher, 1> <went, 1> <to, 1> <the, 1> <store, 1> <the, 1> <store, 1> <was, 1> <closed, 1> <the, 1> <store, 1> <opens, 1> <in, 1> <the, 1> <morning, 1> <the 1> <store, 1> <opens, 1> <at, 1> <9am, 1>
- The output would be:
 - <The, 6> <teacher, 1> <went, 1> <to, 1> <store, 3> <was, 1> <closed, 1> <opens, 1> <morning, 1> <at, 1> <9am, 1>

Combiner (Final Result)

- Essentially an intermediate reducer
- Is optional
- Reduces output from each mapper, reducing bandwidth and sorting
- Cannot change the type of its input
 - Input types must be the same as output types

Output Committer

- Is responsible for taking the reduce output, and committing it to a file
- Typically, this committer needs a corresponding input splitter (so that another job can read the input)
- Again, usually built in splitters are good enough, unless you need to output a special kind of file

Master

- Responsible for scheduling & managing jobs
- Scheduled computation should be close to the data if possible
 - Bandwidth is expensive! (and slow)
 - This relies on a Distributed File System (GFS / HDFS)!
- If a task fails to report progress (such as reading input, writing output, etc), crashes, the machine goes down, etc, it is assumed to be stuck, and is killed, and the step is re-launched (with the same input)
- The Master is handled by the framework, no user code is necessary

Master Cont.

- HDFS can replicate data to be local if necessary for scheduling
- Because our nodes are (or at least should be) deterministic
 - The Master can restart failed nodes
 - Nodes should have no side effects!
 - If a node is the last step, and is completing slowly, the master can launch a second copy of that node
 - This can be due to hardware issues, network issues, etc.
 - First one to complete wins, then any other runs are killed

Outline

- Problem Statement / Motivation
- An Example Program
- MapReduce vs Hadoop
- GFS / HDFS
- MapReduce Fundamentals
- **Example Code**
- Workflows
- Conclusion / Questions

Mapper Code

```
1. public void map(LongWritable key, Text value, Context context) {  
2.     String line = value.toString();  
3.     for(String part : tokenizeString(line)) {  
4.         context.write(new Text(part), new LongWritable(1));  
5.     }  
6. }
```

Our **input** to our mapper is **<LongWritable, Text>**

- The key (the LongWritable) can be assumed to be the position in the document our input is in. This doesn't matter for this example.

Our **output** is a bunch of **<Text, LongWritable>**

- The key is the token, and the value is the count. This is always 1.

Reducer Code

```
1. public void reduce(Text key, Iterable<LongWritable> values, Context context) {  
2.     long sum = 0;  
3.     for (LongWritable val : values) {  
4.         sum += val.get();  
5.     }  
6.     context.write(key, new LongWritable(sum));  
7. }
```

Our **input** is the output of our Mapper, a **<Text, LongWritable>** pair

Our **output** is still a **<Text, LongWritable>**, but it reduces N inputs for token T, into one output <T, N>

Combiner Code

- Do we need a combiner?
 - No, but it reduces bandwidth.
- Our reducer can actually be our combiner in this case.

That's it!

- All that is needed to run the above code is an extremely simple runner class.
 - Simply specifies which components to use, and your input/output directories

Conclusion

- MapReduce provides a simple way to scale your application
- Scales out to more machines, rather than scaling up
- Effortlessly scale from a single machine to thousands
- Fault tolerant & High performance
- If you can fit your use case to its paradigm, scaling is handled by the framework

Conclusion (Cont.)

- Hadoop is a family of open-source products and technologies managed by the Apache Software Foundation (ASF)
- The open-source library of Apache Hadoop is available on the ASF website at apache.org. For users seeking a package that is more ready for enterprise-level use, several vendors offer Hadoop distributions that include additional administrative tools and technical support
- Hadoop is an ecosystem, not a single product
- HDFS is a file system, not a database management system

Conclusion (Cont.)

- Hadoop and MapReduce are related to each other, but they are not mutually dependent.
- MapReduce provides control over analytics, not just analytics alone
- Hadoop is about data diversity, not just about data volume
- Hadoop complements Data Warehouse; rarely used as a substitute for Data Warehouse.
- Hadoop enables various types of analytics, not just web analytics

Latihan Soal

Problem 1

- In a MapReduce framework consider the HDFS block size is 64MB. We have 4 files 64KB, 65MB, 127MB, and 650MB. How many blocks will be created by Hadoop framework?

Problem 1 - Result

- 64KB = 1 block
- 65MB = 2 block
- 127MB = 2 block
- 650MB = 11 block

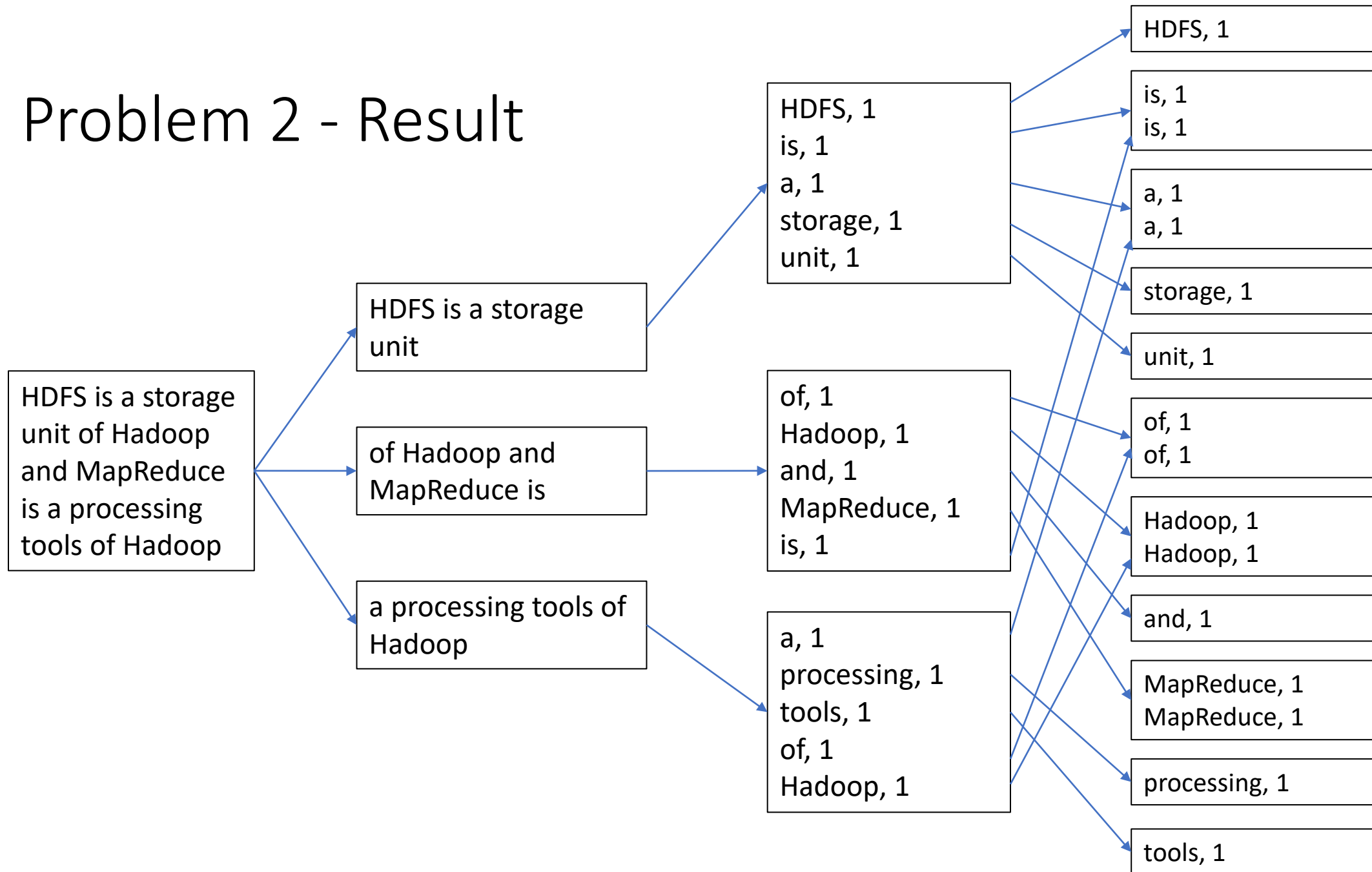
Problem 2

- We have a sentence of words: “HDFS is a storage unit of Hadoop and MapReduce is a processing tools of Hadoop”.
- We need to know the count of each word by using MapReduce. Show the process diagram!

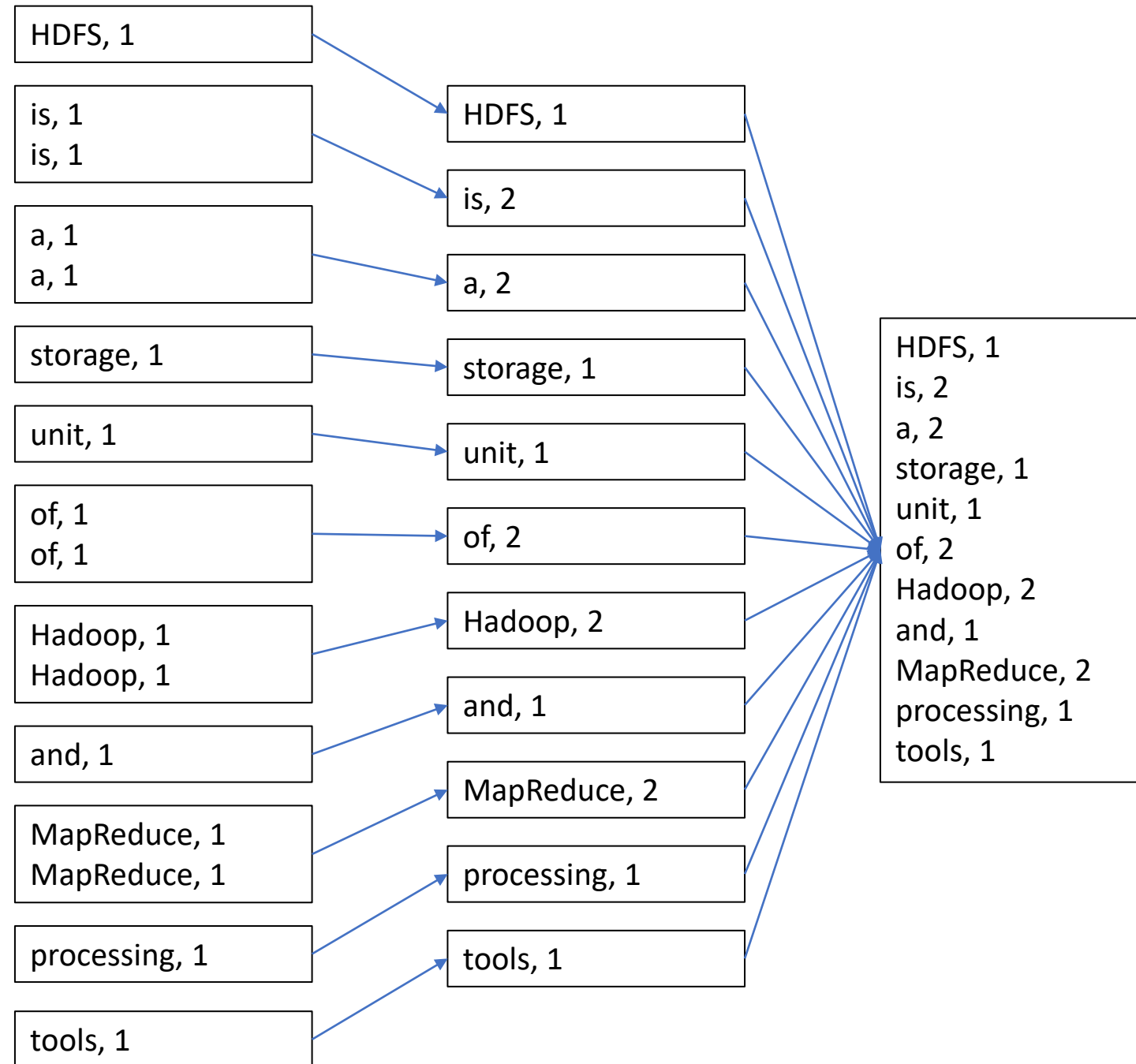
The overall MapReduce word count process

Input Splitting Mapping Shuffling Reducing Final result

Problem 2 - Result



Problem 2 - Result



Problem 3

- This is the table of the XYZ company's employees salary in Oct 2023.
- Draw the process diagram of MapReduce to find the average salary of this following table!

NAME	SALARY (USD)
Random Names	4.200
Peggy Finch	4.700
Henrietta Mueller	6.000
Maya Jennings	6.600
Leanne Hill	3.400
Wyatt Harrell	3.600
Sonny Bradshaw	4.900
Ismail Ortiz	4.600
Terence Ochoa	4.200
Cindy Conner	2.200
Tyler Glenn	6.600

Problem 3 - Result

- Sama seperti problem 2 dengan penambahan fungsi average pada saat proses reduce.

Problem 4

- Write the pseudocode of Problem 1, 2, and 3!

Problem 4 - Result

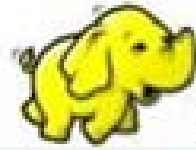
Algorithm 3.1 Word count (repeated from Algorithm 2.1)

The mapper emits an intermediate key-value pair for each word in a document.
The reducer sums up all counts for each word.

```
1: class MAPPER
2:   method MAP(docid  $a$ , doc  $d$ )
3:     for all term  $t \in \text{doc } d$  do
4:       EMIT(term  $t$ , count 1)

1: class REDUCER
2:   method REDUCE(term  $t$ , counts  $[c_1, c_2, \dots]$ )
3:      $sum \leftarrow 0$ 
4:     for all count  $c \in \text{counts } [c_1, c_2, \dots]$  do
5:        $sum \leftarrow sum + c$ 
6:     EMIT(term  $t$ , count  $sum$ )
```

Extra



Apache Hadoop Ecosystem



Ambari

Provisioning, Managing and Monitoring Hadoop Clusters



Sqoop

Data Exchange



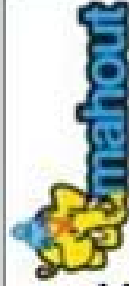
Oozie

Workflow



Pig

Scripting



Mahout

Machine Learning

R Connectors

Statistics



Hive

SQL Query



Hbase

Columnar Store



Flume

Log Collector

Zookeeper

Coordination



HDFS

Hadoop Distributed File System

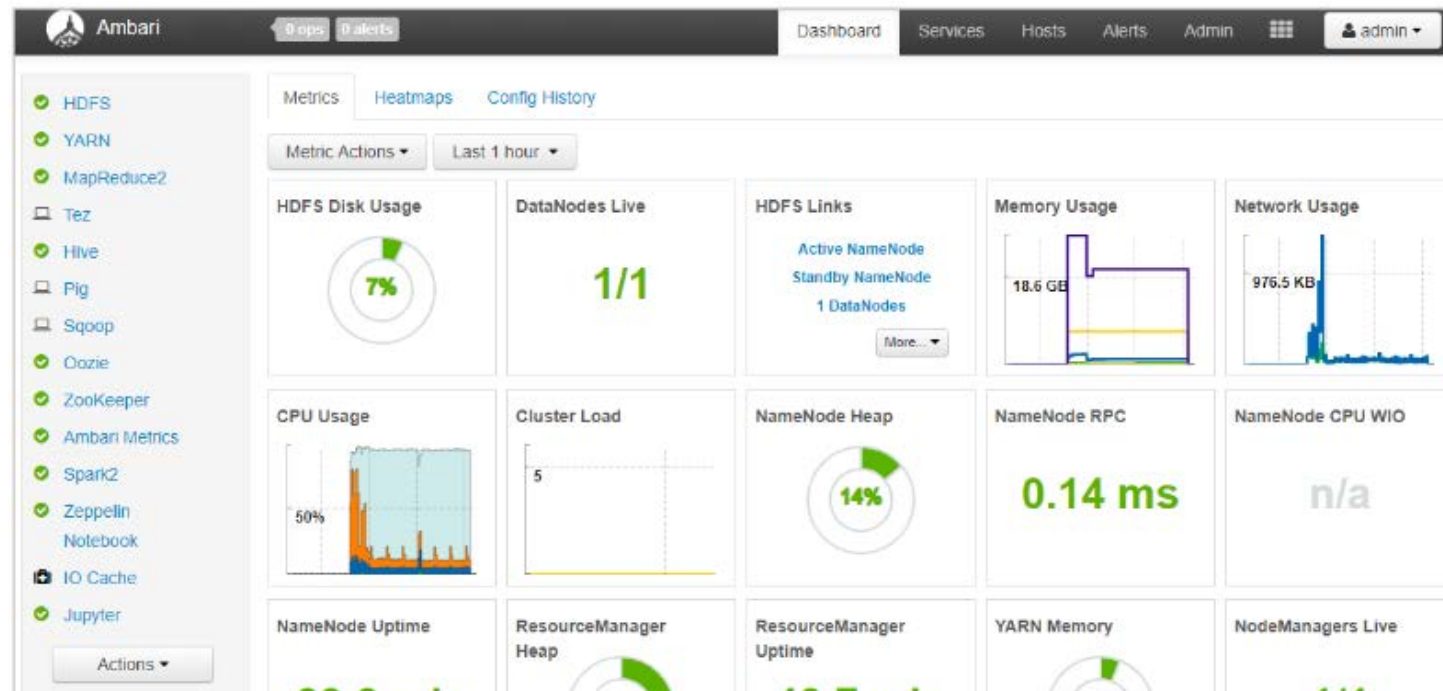
YARN Map Reduce v2

Distributed Processing Framework



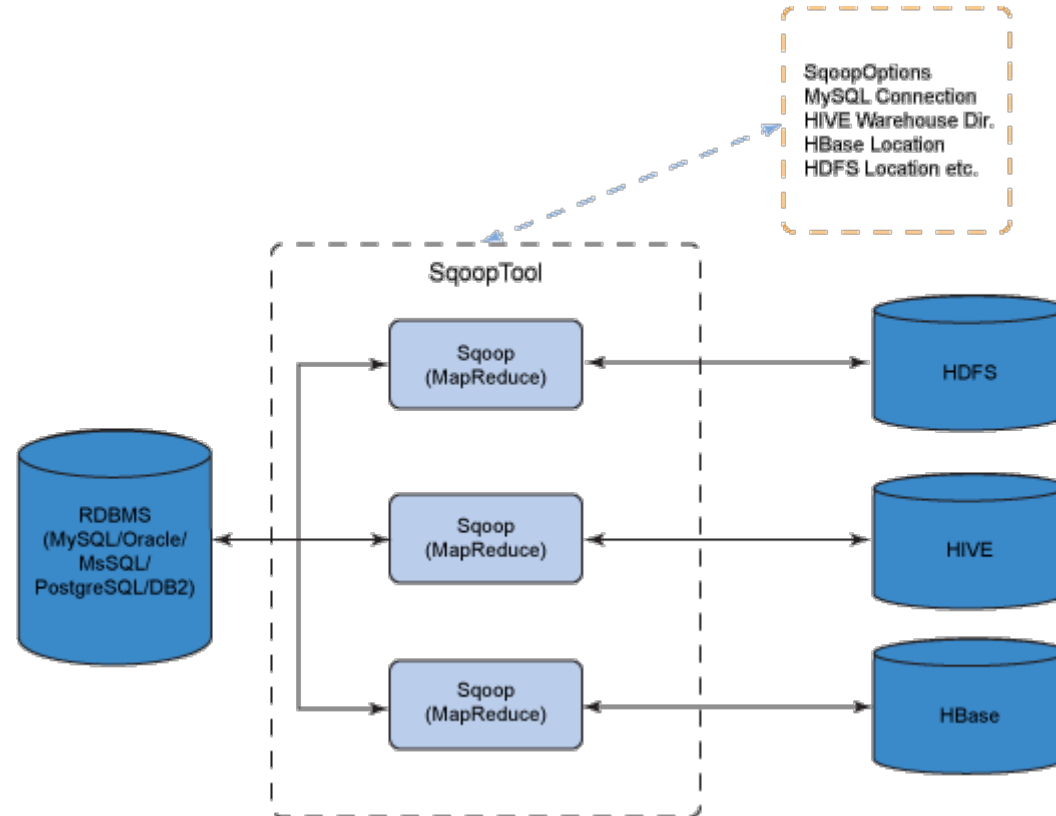
Apache Hadoop Ecosystem – (Cont)

- Apache Ambari
 - is an open-source administration tool deployed on top of Hadoop clusters, and it is responsible for keeping track of the running applications and their status. Apache Ambari can be referred to as a web-based management tool that manages, monitors, and provisions the health of Hadoop clusters.



Apache Hadoop Ecosystem – (Cont)

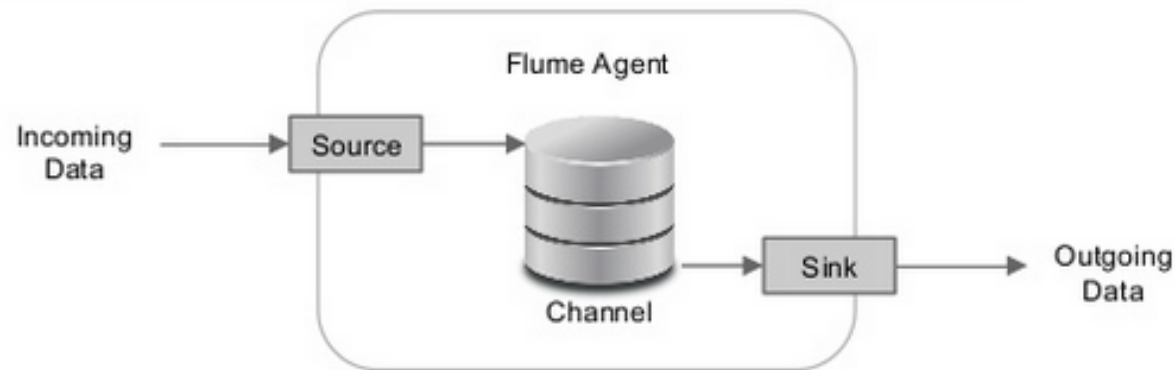
- Apache Sqoop
 - Apache Sqoop is a tool designed for efficiently transferring bulk data between and structured datastores such as relational databases.



Apache Hadoop Ecosystem – (Cont)

- Apache Slume

- Flume is a distributed, reliable, and available service for efficiently collecting, aggregating, and moving large amounts of log data.



Source

- Accepts incoming Data
- Scales as required
- Writes data to Channel

Channel

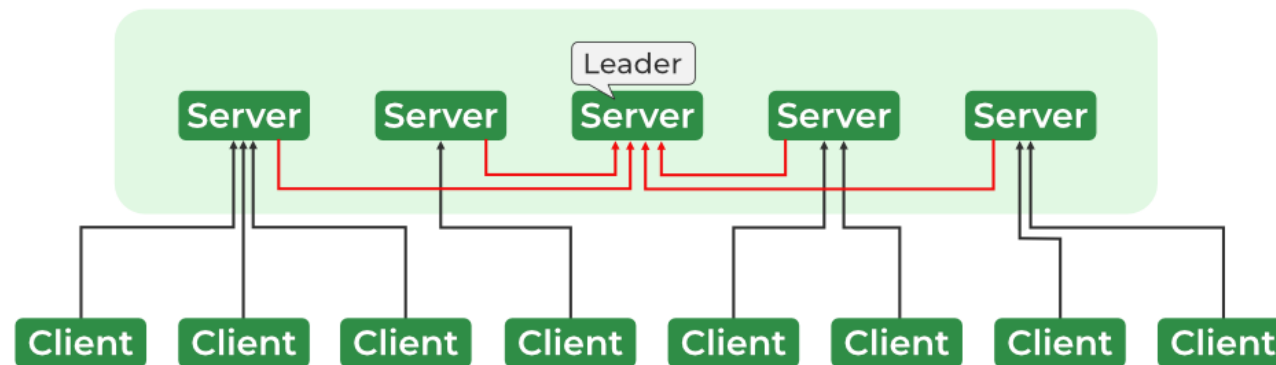
- Stores data in the order received

Sink

- Removes data from Channel
- Sends data to downstream Agent or Destination

Apache Hadoop Ecosystem – (Cont)

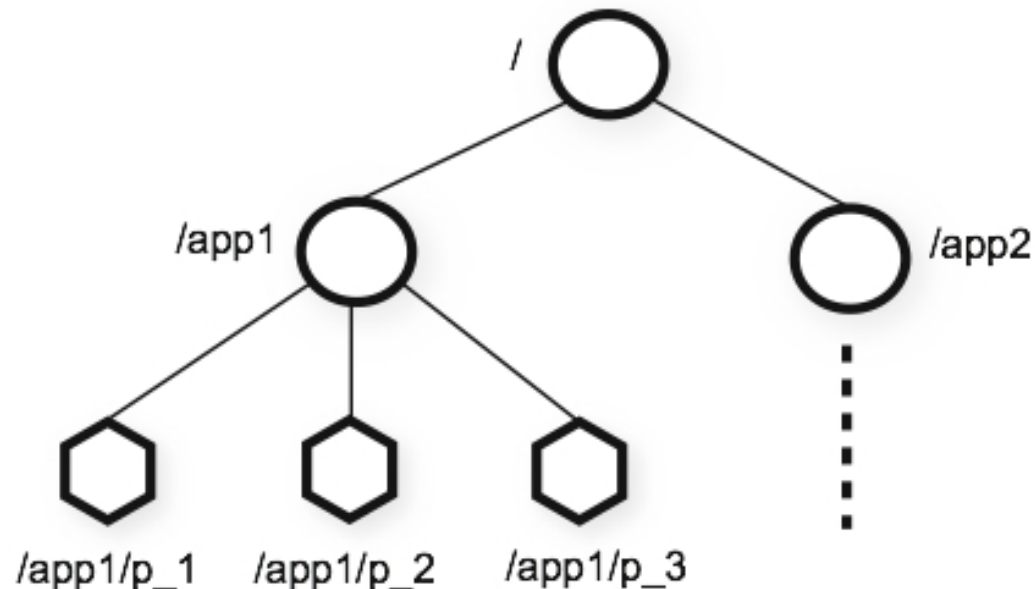
- Zookeeper
 - ZooKeeper is a distributed, open-source coordination service for distributed applications. ZooKeeper allows distributed processes to coordinate with each other through a shared hierarchal namespace which is organized similarly to a standard file system.



Apache Hadoop Ecosystem – (Cont)

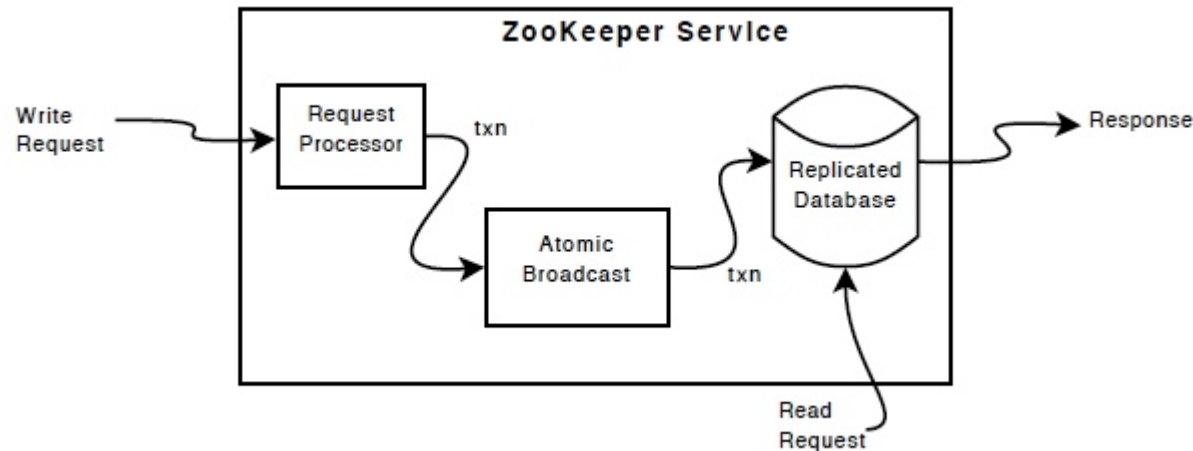
ZooKeeper's Hierarchical Namespace

- Zookeeper (Cont.)
 - ZooKeeper is a distributed, open-source coordination service for distributed applications. ZooKeeper allows distributed processes to coordinate with each other through a shared hierarchal namespace which is organized similarly to a standard file system



Apache Hadoop Ecosystem – (Cont)

- Zookeeper (cont.)



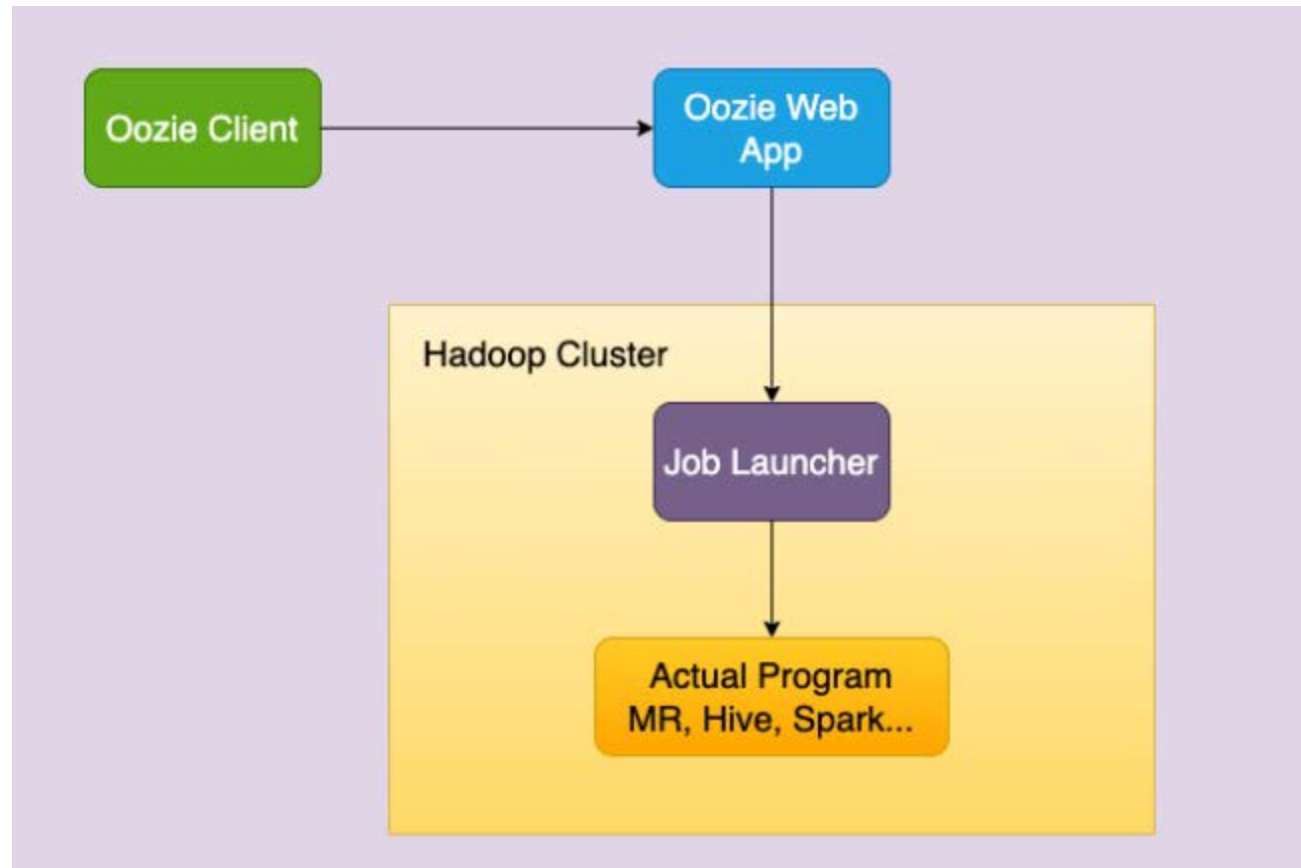
Request Processor – Active in Leader Node and is responsible for processing write requests. After processing, it sends changes to the **follower nodes**

Atomic Broadcast – Present in both Leader Node and Follower Nodes. It is responsible for sending the changes to other Nodes.

Replicated Databases - It is responsible for storing the data in the zookeeper. Every node contains its own databases. Data is also written to the file system providing recoverability in case of any problems with the cluster.

Apache Hadoop Ecosystem – (Cont)

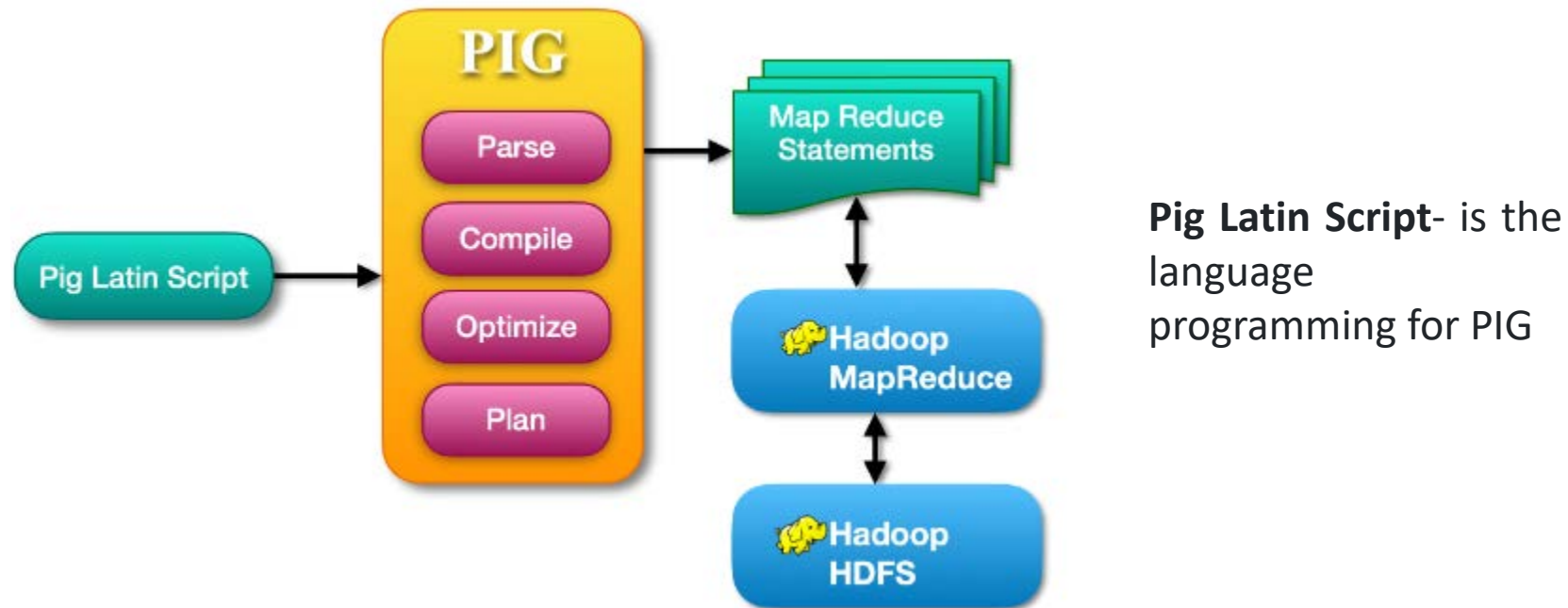
- Oozie
 - Oozie is a workflow scheduler system to manage Apache Hadoop jobs.



Apache Hadoop Ecosystem – (Cont)

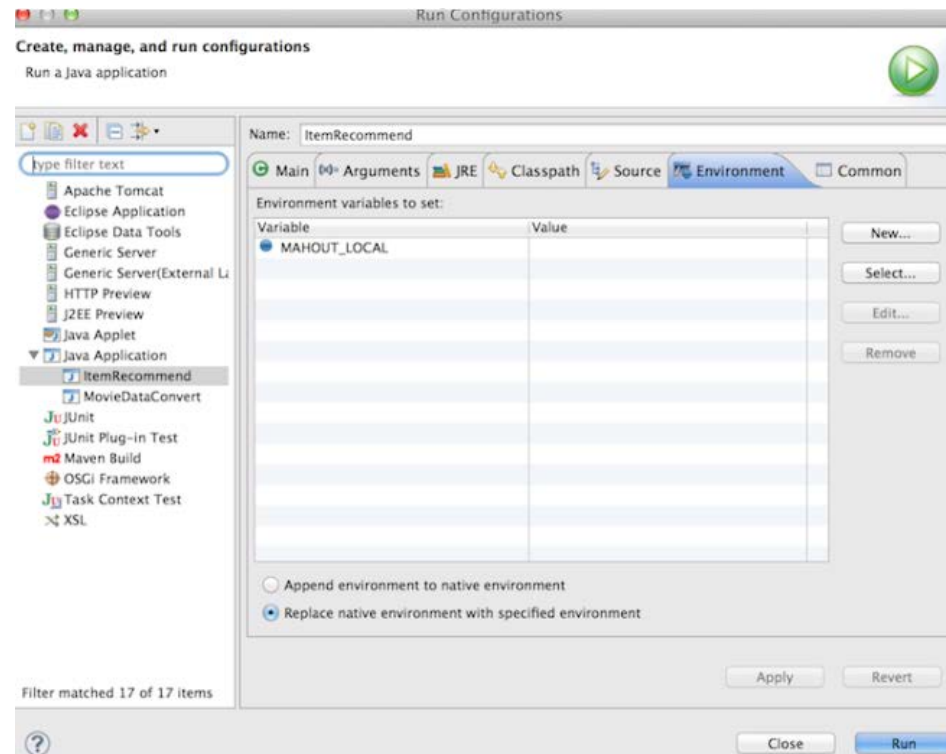
- Apache Pig

- is a platform for analyzing large data sets that consists of a high-level language for expressing data analysis programs, coupled with infrastructure for evaluating these programs.



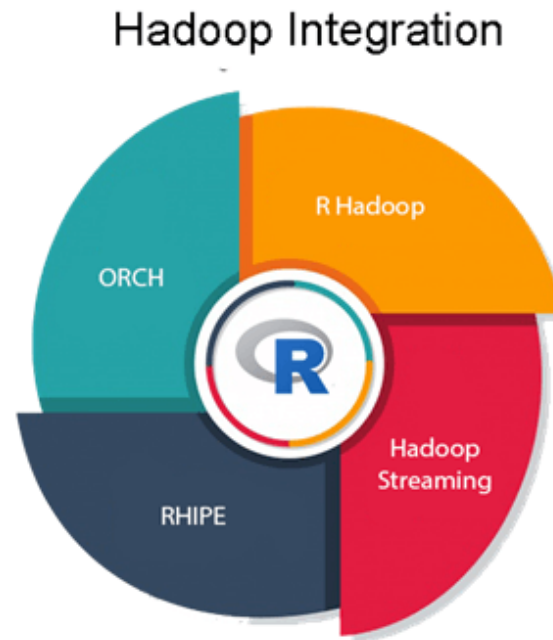
Apache Hadoop Ecosystem – (Cont)

- Apache Mahout
 - Apache Mahout is a **distributed linear algebra framework** and **mathematically expressive Scala DSL** designed to let mathematicians, statisticians, and data scientists quickly *implement their own algorithms*.
 - Recommended out-of-the-box distributed back-end



Apache Hadoop Ecosystem – (Cont)

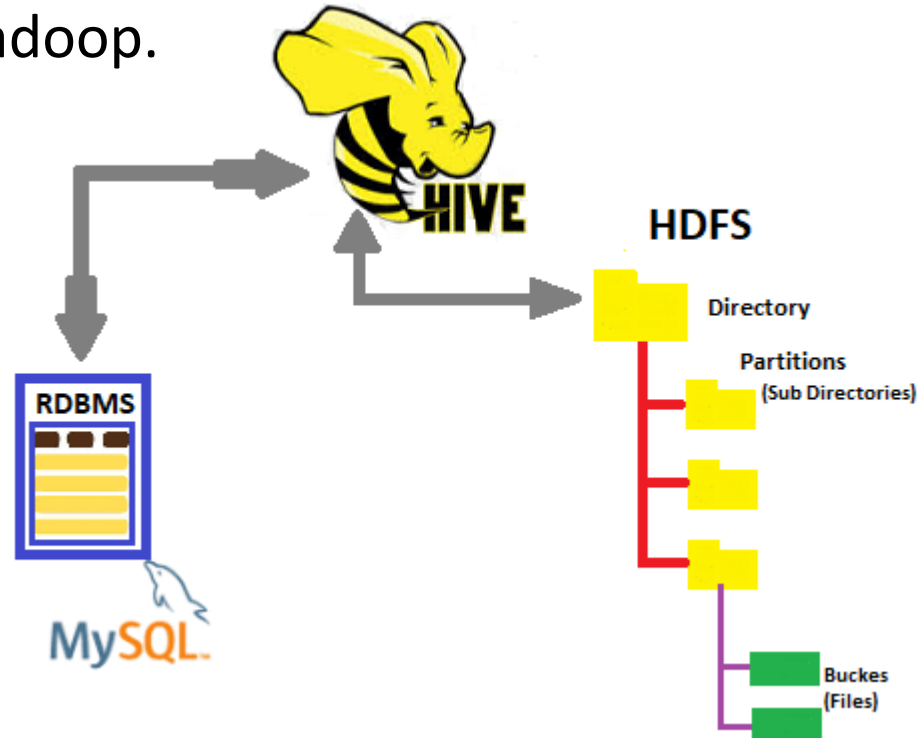
- R connector
 - “R” = Statistical Modelling language.
 - R and Hadoop were not natural friends.
 - Needs “connector” as Rhadoop, RHIVE, and RHIPE



Apache Hadoop Ecosystem – (Cont)

- Apache Hive

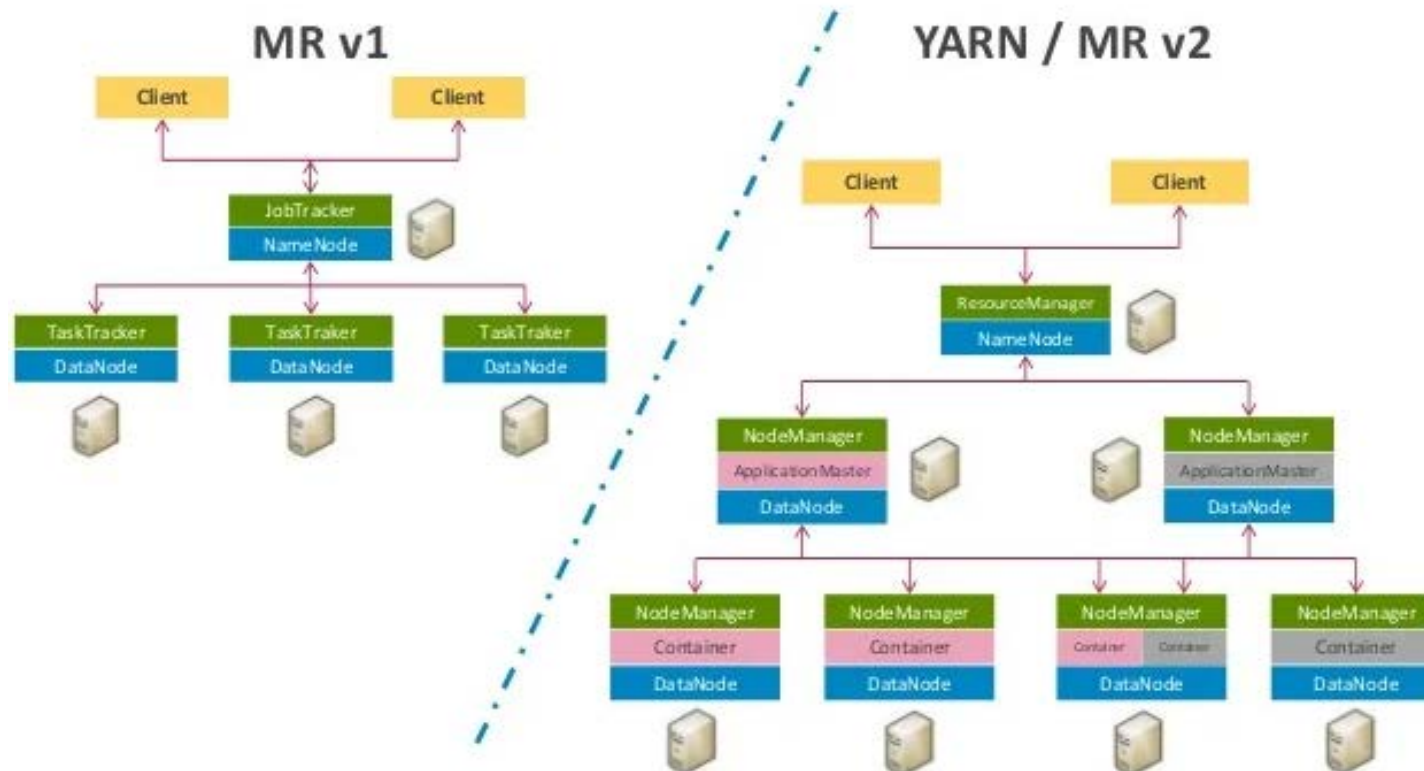
- Apache Hive is a data warehouse software project built on top of Apache Hadoop for providing data query and analysis
- Hive gives an SQL-like interface to query data stored in various databases and file systems that integrate with Hadoop.



Apache Hadoop Ecosystem – (Cont)

- **YARN (Yet Another Resource Negotiator) MapReduce v2** = next-gen version of MapReduce

MR VS. YARN ARCHITECTURE



- **YARN** : Yet Another Resource Negotiator
- **MR** : MapReduce

Apache Hadoop Ecosystem – (Cont)

- **Hbase** = Hyper NoSQL Database
 - Distributed column-oriented database built on top of the Hadoop file system
 - Non-relational database that enables short and low-latency data searches in Hadoop
 - Allows for transactional operations in Hadoop, including updates, inserts, and deletes.

